

Motion Analysis & Tracking

Lectures on Image Processing

Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2, 612 66 Brno - Královo Pole



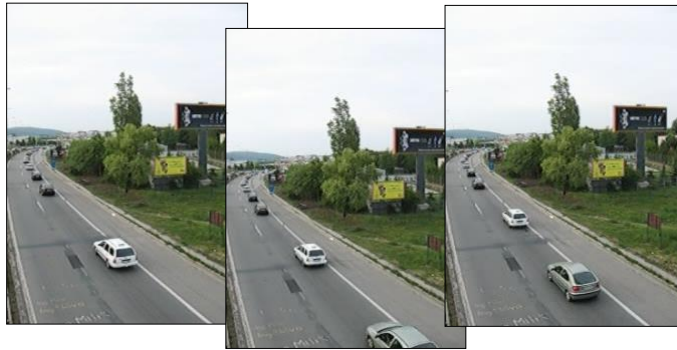
*Change detection and object tracking
are key elements of computer vision!“*



- Motivation and Examples
- Typical problems
- Differential methods
- Background modelling
- Optical flow
- KLT tracker

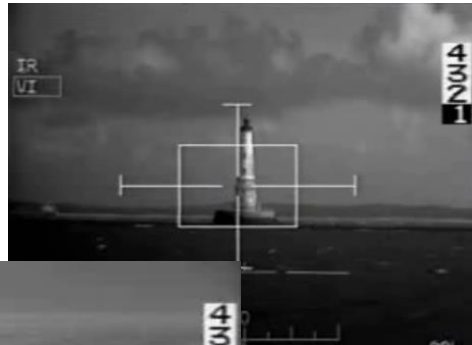
- Šonka, M., Hlaváč, V., Boyle, R.: Image Processing, Analysis and Machine Vision, 2ed, Brooks and Cole Publishing, 1998.
- Hlaváč, V.: Image Motion, přednáška, CMP, FEL, ČVUT v Praze.
- Forsyth, Ponce: Computer Vision A Modern Approach, Prentice Hall, 2003.

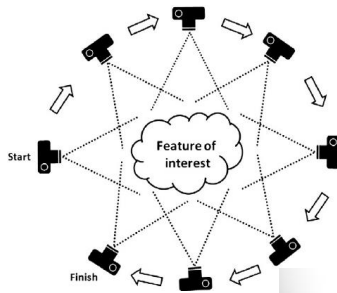
- Function $F(x, y, t)$
 - x, y ... spatial coordinates
 - t ... time, index of the video frames

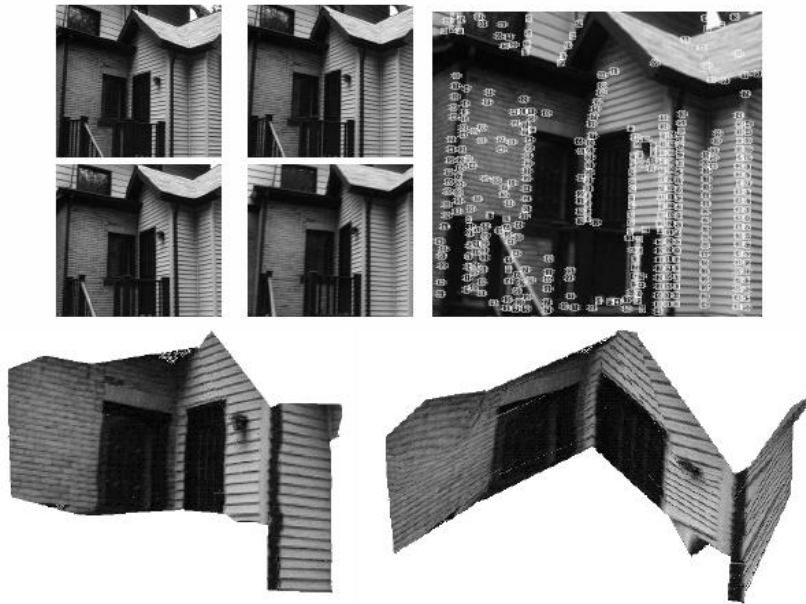


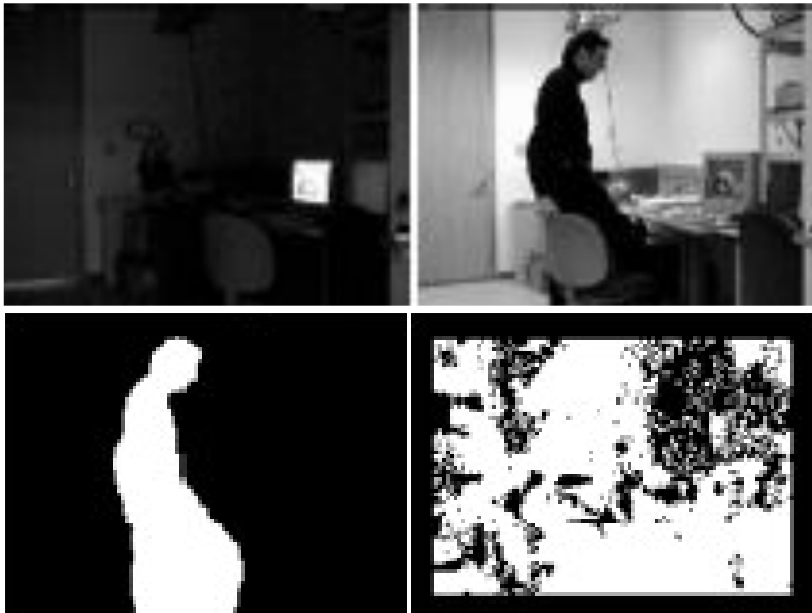


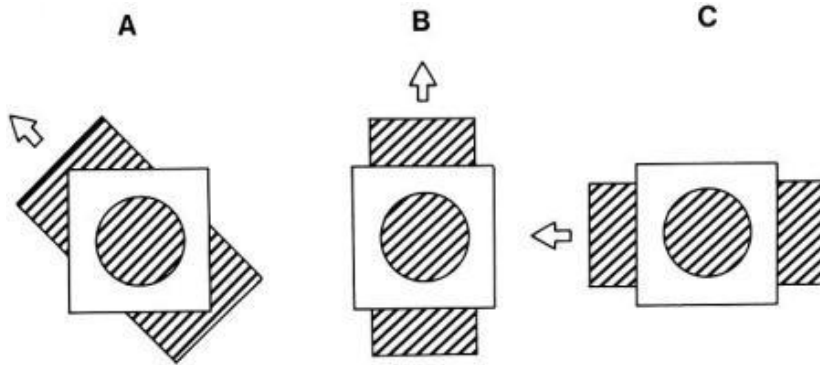


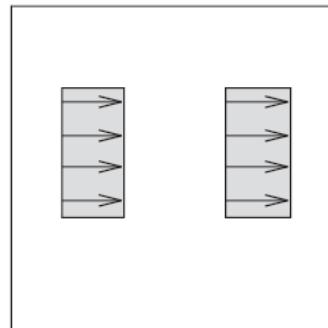
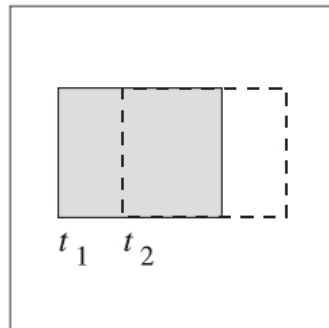


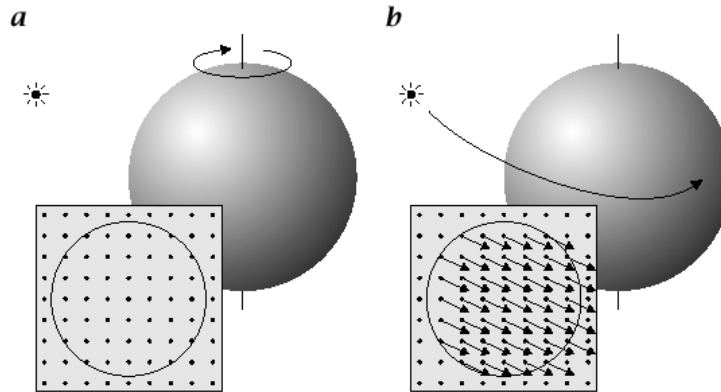














Problems with low precision of object localization, problems with classification etc.

- Difference of consecutive frames

$$d(i, j) = \begin{cases} 0 & \text{if } |f_1(x, y) - f_2(x, y)| \leq \varepsilon \\ 1 & \text{otherwise} \end{cases}$$



- Reference frame update

- Average

$$I_{\text{Ref}}(x, y, t) = \frac{1}{N} \sum_{i=1}^N I(x, y, t-i)$$

- Sliding window

$$I_{\text{Ref}}(x, y, t+1) = \alpha I_{\text{Ref}}(x, y, t) + (1-\alpha)I(x, y, t)$$

Simple and fast calculation.

Very sensitive to noise, slight movements, etc.

-> often supplemented by further processing or detection of blobs.

Changing the lighting has a significant effect on the result

-> reference image update.



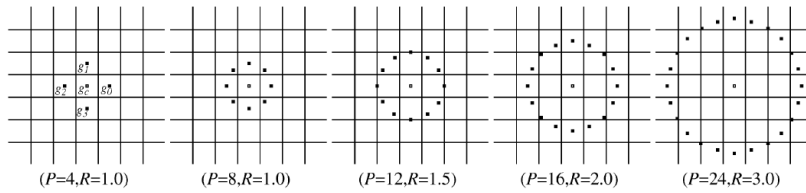
Simple and fast calculation.

Very sensitive to noise, slight movements, etc.

-> often supplemented by further processing or detection of blobs.

Changing the lighting has a significant effect on the result

-> reference image update.

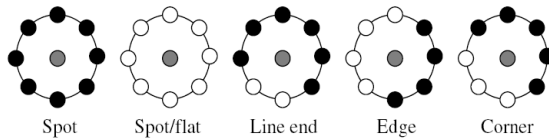


example	thresholded	weights																											
<table border="1"> <tr><td>6</td><td>5</td><td>2</td></tr> <tr><td>7</td><td>6</td><td>1</td></tr> <tr><td>9</td><td>8</td><td>7</td></tr> </table>	6	5	2	7	6	1	9	8	7	<table border="1"> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	1	0	0	1	0	0	1	1	1	<table border="1"> <tr><td>1</td><td>2</td><td>4</td></tr> <tr><td>128</td><td>0</td><td>8</td></tr> <tr><td>64</td><td>32</td><td>16</td></tr> </table>	1	2	4	128	0	8	64	32	16
6	5	2																											
7	6	1																											
9	8	7																											
1	0	0																											
1	0	0																											
1	1	1																											
1	2	4																											
128	0	8																											
64	32	16																											

Pattern = **11110001**
LBP = $1 + 16 + 32 + 64 + 128 = 241$

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p,$$

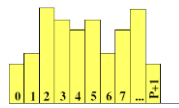
$$s(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



Ojala, T., Pietikäinen, M., Mäenpää, T.: Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns, 2002.

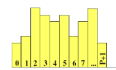
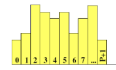
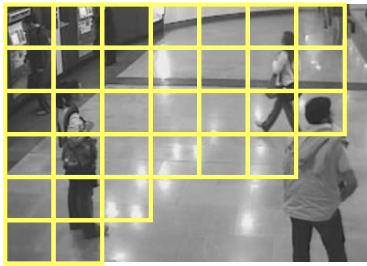


LBP_{P,R}

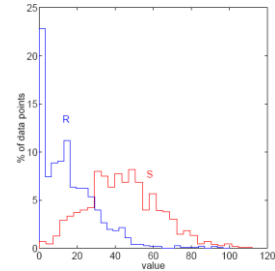
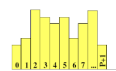


Chi-squared distance for histogram comparison

$$\chi^2 = \sum_i \frac{(R_i - S_i)^2}{R_i + S_i}$$



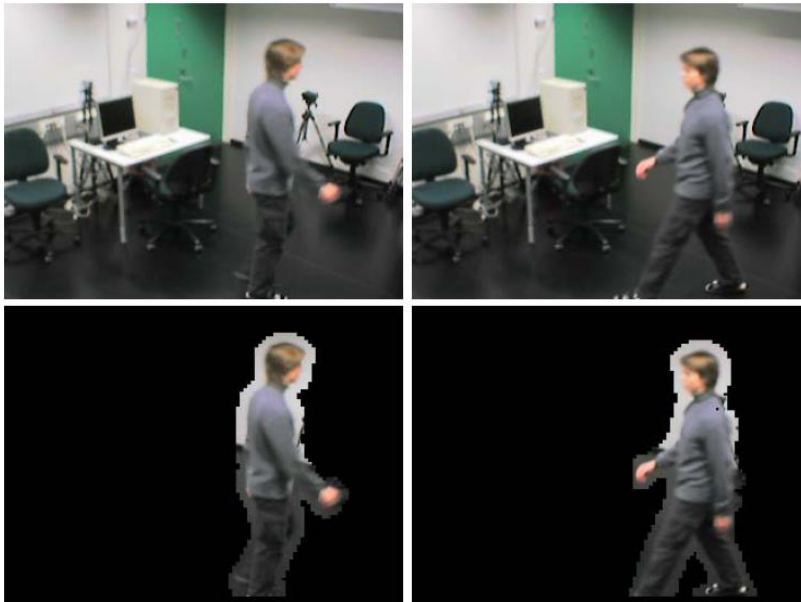
...



Histogram of all LBP values in the image region.

Movement detection – compare histograms of actual and reference (or previous) frames.

Image split into overlapping regions – leads to better localization precision.



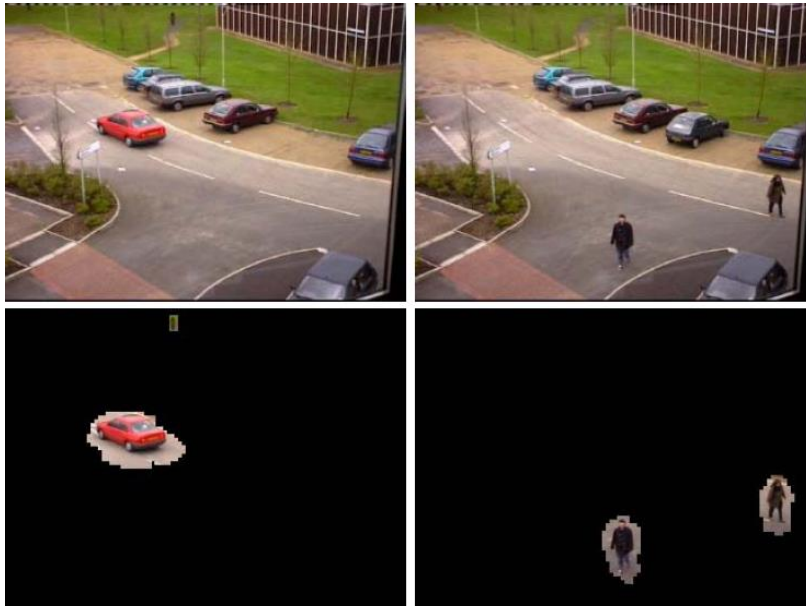
Fast calculation, easy implementation in HW.

Invariant to pixel intensity transformations in the image (eg lighting changes).

Stable against minor camera shake.

Inaccurate location of changes

Only detection of changes in the image, unknown direction



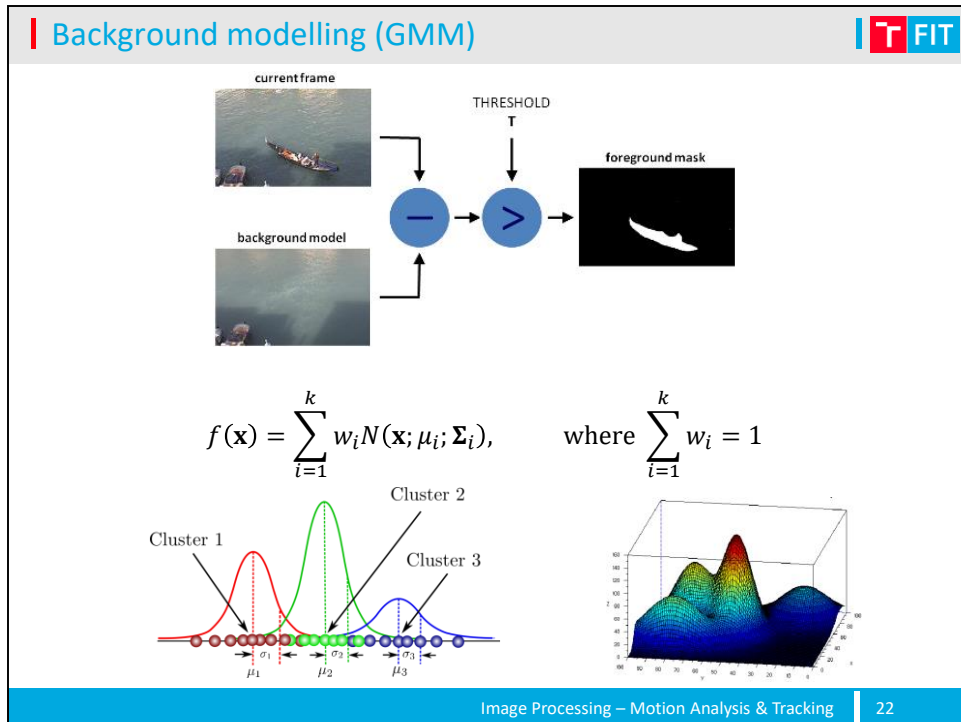
Fast calculation, easy implementation in HW.

Invariant to pixel intensity transformations in the image (eg lighting changes).

Stable against minor camera shake.

Inaccurate location of changes

Only detection of changes in the image, unknown direction

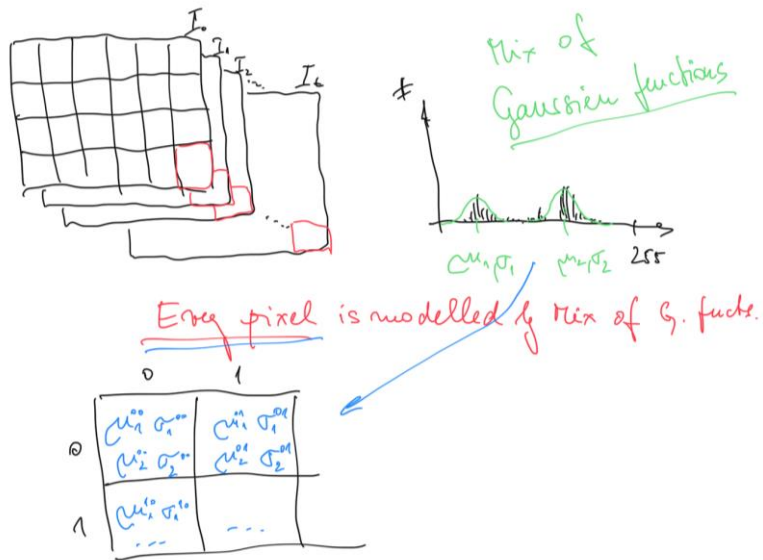


Segmentation of foreground from the background.

Simple model of the background (e.g. Average frame or Reference frame) is not good enough.

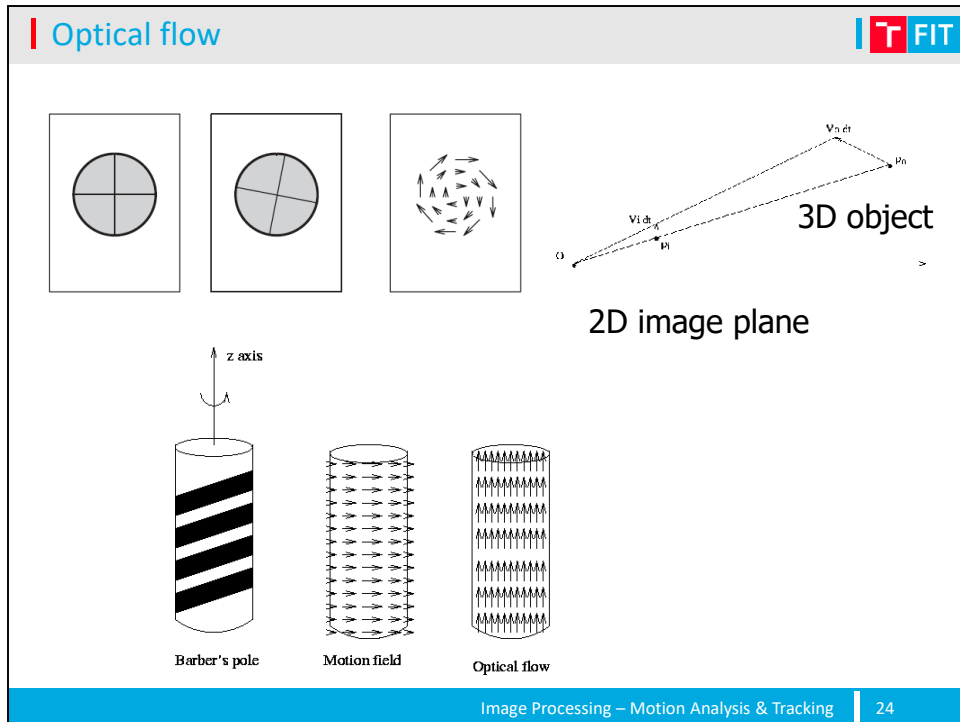
Gaussian Mixture Model (GMM)

Linear combination of Gaussian functions.



Each image pixel modelled by 3-5 Gaussian functions.

Expectation Maximization is a method fitting the Gaussian functions to data distribution.

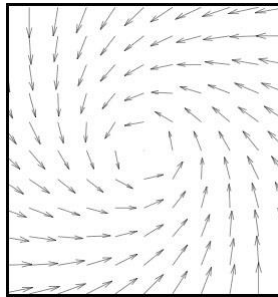
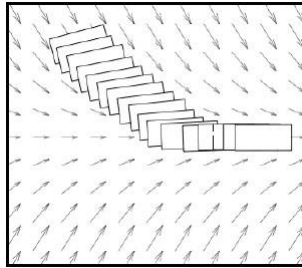


Optical flow is the apparent motion of brightness patterns in the image.

Motion field describes the movement of 3D objects projected to 2D image – vectors of the movements.

Sensitive to window size, lighting changes, shadows etc.

In general, optical flow corresponds to the motion field. But not always (see Barbers pole). But this is very unusual, so we keep that ... optical flow corresponds to the motion field.



- Differential methods
 - Image derivations
 - Horn-Shunck
 - Lucas-Kanade
- Matching methods
 - Detection and tracking of specific structures (features)

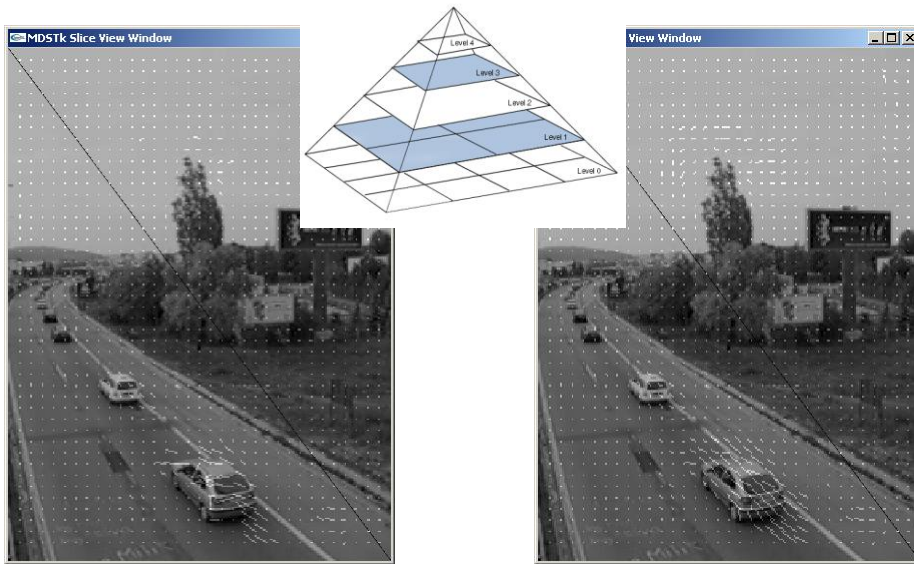
- Standard equation of optical flow
 - $I_x u + I_y v + I_t = 0$
 - I_x, I_y, I_t ... image derivatives
 - u, v ... optical flow (vector of movements)
- Gradient equation
 - $(\nabla I)^T \cdot f = -I_t$
 - $f = (u, v)$... optical flow
 - $\nabla I = (I_x, I_y)$... gradient of image function
- Error function
 - $E(f) \approx \sum_{x,y \in R} [\nabla I(x, y, t) \cdot f + I_t(x, y, t)]^2$
- Least square method + math.
 - $f = -A^{-1}b$
 - $A = \sum_{x,y \in R} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}^2 \quad b = \sum_{x,y \in R} \begin{bmatrix} I_x I_t \\ I_y I_t \end{bmatrix}$

The calculation can only be used to move in the fractional pixel range -
> iterative version of the algorithm.

Estimation of optical flow $f(u, v)$ between images $I(t)$ and $I(t+1)$.

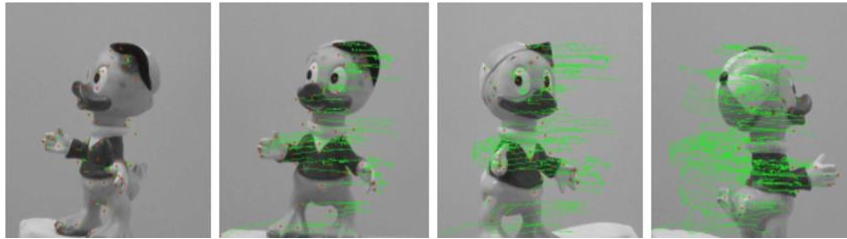
Iteration

- The image $I(x, y, t)$ is shifted according to the optical flow obtained so far.
- Estimation of residual motion df for a "shifted" image and $I(x, y, t+1)$ -> $f' = f + df$



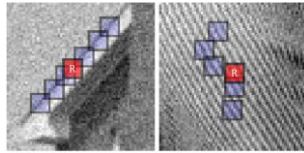
Often extended by using an image pyramid -> it captures fast movements better (at the cost of more frequent large errors).

We start on the top floor of the pyramid = the most subsampled image.



Find correspondences:

- SSD - Sum of Squared Differences
- Feature descriptors comparison (computed from feature's local region)



$$E(x, y; d) = \sum_{(x', y') \in N(x, y)} [I_L(x' + d, y') - I_R(x', y')]^2$$

Matching approach.

Extract (visual) features (keypoints, corners, blobs)

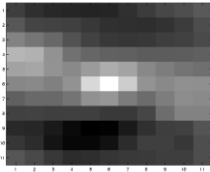
Matching (track) between frames

More robust motion tracking, especially for movement in the pixel range up to tens of pixels.

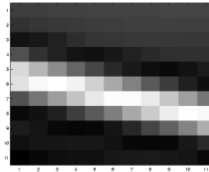
Find correspondences:

1. SSD - Sum of Squared Differences
2. Feature descriptors comparison (computed from feature's local region)

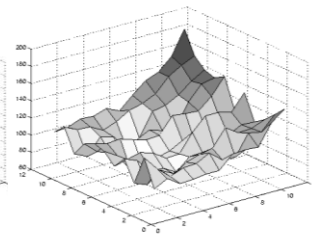
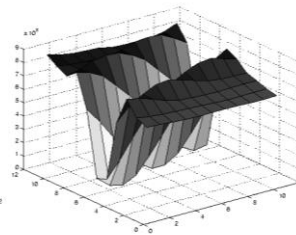
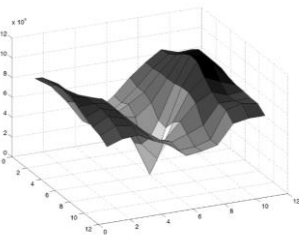
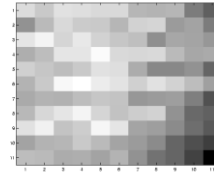
texture

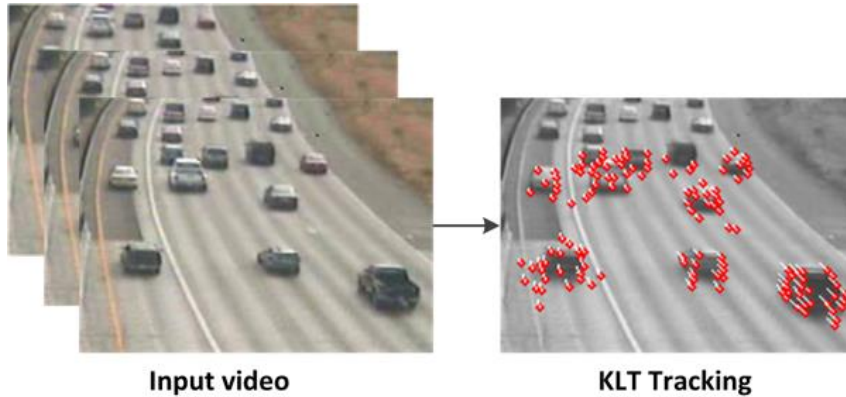


edge



homogenous





Matching approach.

- Extract (visual) features (keypoints, corners, blobs) (e.g. Shi, Tomasi: Good features to track (**Harris corners**))
- Match them (track) between frames (e.g. Lucas-Kanade optical flow + check features (points) consistency and add new features)

More robust motion tracking, especially for movement in the pixel range up to tens of pixels.

- Kalman filters
- Particle filters
 - The state (position) of an object is modeled by a set of particles with assigned weights
 - Importance sampling step
 - Selection step

